

Lab 04 — Praktijklab: de image van de API bouwen

Doel: zelf de Dockerfile van een (nep-)Spring Boot-API schrijven en elke valkuil uit de theorie zelf uitlokken — context, cache, `CMD`, `ENTRYPOINT`, shell-vorm, `USER` — op een build-engine zonder daemon.

Vereisten — Labs 01 tot 03 afgewerkt.

Geleverde bestanden (`files/`) - `Api.java` — een HTTP-API van 30 regels, zonder afhankelijkheden. Ze serveert `/` en `/actuator/health`, leest `APP_MESSAGE`, `APP_PROFILE` en `SERVER_PORT` uit de omgeving, en vangt `SIGTERM` op. **Je hoeft er nooit iets aan te wijzigen:** deze labs gaan over containers, niet over Java. - `construire-jar.sh` — compileert `Api.java` naar `api.jar` in een wegwerptainer, zodat je geen JDK op je WSL hoeft te installeren.

Stap 1 — Het project voorbereiden

```
mkdir -p ~/labo-docker/04 && cd ~/labo-docker/04
cp <pad-van-het-lab>/files/Api.java .
cp <pad-van-het-lab>/files/construire-jar.sh . && chmod +x construire-jar.sh
./construire-jar.sh
```

Observeer de download van `docker.io/library/eclipse-temurin:21-jdk` (eenmalig, ~490 MB), gevolgd door een `api.jar` van zo'n 2,4 KB.

Uitleg. Je hebt net een container als **wegwerptool** gebruikt: de compilatie draaide in een volledige JDK, gemount op je map, en daar blijft niets van achter. Dat is meteen ook het basisidee van de multi-stage-builds in lab 05.

→ JAVA

`javac` compileert een `.java` -bestand naar `.class` -bestanden (*bytecode*, machine-onafhankelijk); `jar` verpakt die `.class` -bestanden in een ZIP-archief met een manifest dat aangeeft welke klasse gestart moet worden. De **JDK** (*Development Kit*) bevat die tools; de **JRE** (*Runtime Environment*) bevat alleen wat nodig is om het resultaat *uit te voeren* — daarom is die maar half zo groot, en daarom kiezen we hem straks voor de uiteindelijke image.

Stap 2 — Een eerste Dockerfile

Maak `Dockerfile` aan:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
podman build -t api-lab:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-lab|temurin'
```

Observeer de stappen STEP 1/6 tot STEP 6/6, elk gevolgd door een identificatie --> ..., daarna COMMIT api-lab:1.0 en Successfully tagged localhost/api-lab:1.0. De image weegt **209 MB** — voor een JAR van 2,4 KB. Vrijwel alles daarvan is de JRE.

```
podman run -d --name api -p 18080:8080 api-lab:1.0
curl -s localhost:18080/ ; echo
curl -s localhost:18080/actuator/health ; echo
podman logs api
podman port api
```

Observeer {"message":"Bonjour depuis l'API","profile":"default"}, {"status":"UP"}, de logregels Arguments recus : --spring.profiles.active=prod en API demarree sur le port 8080 (profil default), en 8080/tcp -> 0.0.0.0:18080.

Uitleg. EXPOSE 8080 heeft niets gepubliceerd; de doorsturing komt van -p 18080:8080. Bewijs het zelf: start podman run -d --name api2 api-lab:1.0 en zie curl -m 2 localhost:18080/ mislukken... en denk eraan dat een -p 80:8080 in rootless-modus geweigerd zou worden.

```
podman rm -f -t 0 api api2
```

→ WINDOWS / WSL

Open <http://localhost:18080/actuator/health> in je Windows-browser: WSL stuurt de poort door. In lab 07 test je de Angular-frontend op precies dezelfde manier.

Stap 3 — De build context

```
mkdir -p ../gemeenschappelijk && echo "privesleutel" > ../gemeenschappelijk/secret.txt
printf 'FROM docker.io/library/alpine\nCOPY ../gemeenschappelijk/secret.txt /\n' >
Dockerfile.buiten-context
podman build -f Dockerfile.buiten-context -t poging .
```

Observeer de mislukking: Error: building at STEP "COPY ../gemeenschappelijk/secret.txt /": ... possible escaping context directory error: copier: stat: "/gemeenschappelijk/secret.txt": no such file or directory.

Uitleg. Buildah trok het pad terug tot *binnen* de context (/gemeenschappelijk/secret.txt ten opzichte van de map) en vond daar niets. Dit is geen rechtenprobleem: het bestand valt gewoon buiten de grens.

Meet nu wat een ongefilterde context kost:

```
mkdir -p node_modules && dd if=/dev/zero of=node_modules/groot.bin bs=1M count=200 2>/dev/null
printf 'FROM docker.io/library/alpine\nCOPY . /src\n' > Dockerfile.alles
time podman build -q -f Dockerfile.alles -t api-lab:alles .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep alles
```

Observeer een build van ongeveer 1,7 s... en een image van **218 MB** op een `alpine` van 8,7 MB: de 200 MB uit `node_modules` zit er integraal in.

```
printf 'node_modules\n*.bin\nDockerfile.buiten-context\n' > .dockerignore
time podman build -q -f Dockerfile.alles -t api-lab:alles2 .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep alles
```

Observeer `8.71 MB` voor `alles2`, en een snellere build.

Uitleg. Docker zou die 200 MB eerst naar de daemon **overgedragen** hebben (`transferring context`); Podman leest de map ter plaatse, waardoor de build "snel" aanvoelt. Maar de image neemt nog steeds alles mee wat de `COPY . .` raakt. `.dockerignore` werkt **vóór** elke instructie: zonder dat bestand zou `node_modules` mee in de gepubliceerde image gaan. Podman aanvaardt ook de naam `.containerignore` — zelfde inhoud, zelfde effect.

Stap 4 — De cache, en de volgorde van de instructies

Voeg een gesimuleerde stap "afhankelijkheden" toe. Vervang je `Dockerfile` door:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN echo "afhankelijkheden downloaden..." && sleep 5
COPY api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
time podman build -t api-lab:2.0 .
time podman build -t api-lab:2.0 .
```

Observeer een eerste build van ongeveer 6,4 seconden, en dan een tweede in 0,7 s, met `--> Using cache` onder elke stap.

Wijzig de JAR en bouw opnieuw:

```
touch Api.java && ./construire-jar.sh >/dev/null
time podman build -t api-lab:2.1 .
```

Observeer dat de stap `RUN ... sleep 5` nog altijd `--> Using cache` toont: alleen de `COPY` en alles erna worden opnieuw gedaan. 0,7 s.

Draai nu de volgorde om — de `COPY` **vóór** de `RUN`:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
RUN echo "afhankelijkheden downloaden..." && sleep 5
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -t api-lab:3.0 .
./construire-jar.sh >/dev/null && time podman build -t api-lab:3.1 .
```

Observeer dat je de 5 seconden nu **bij elke build betaalt**: 6,3 s.

Uitleg. Dit is, in het klein, het verschil tussen een Maven-build van 40 seconden en een van 6 minuten: een vluchtige `COPY` vóór de dure stap doet alles erna vervallen.

Stap 5 — `CMD` tegenover `ENTRYPOINT`

Bouw de image van stap 2 opnieuw (ze heeft zowel een `ENTRYPOINT` als een `CMD`) en bekijk hoe die samenwerken:

```
timeout 5 podman run --rm api-lab:1.0 | head -2
timeout 5 podman run --rm api-lab:1.0 --debug | head -2
```

Observeer in het eerste geval `Arguments recurs : --spring.profiles.active=prod`, in het tweede `Arguments recurs : --debug`. De `ENTRYPOINT` (`java -jar ...`) bleef staan; alleen de `CMD` werd vervangen.

Vergelijk met een image die alleen een `CMD` heeft:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCMD ["java","-jar","/app/api.jar"]\n' > D-cmd
podman build -q -f D-cmd -t api-lab:cmd .
podman run --rm api-lab:cmd sh -c 'echo "ik vervang alles"'
```

Observeer dat de API **helemaal niet** start: bij een kale `CMD` vervangt jouw argument het volledige commando.

```
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo "shell verkregen ondanks ENTRYPOINT"'
```

Observeer dat `--entrypoint` de nooduitgang is om te debuggen.

Uitleg. `CMD` is een vervangbare standaardwaarde; `ENTRYPOINT` is een vast programma waar argumenten achteraan komen. Het patroon in bedrijven is `ENTRYPOINT` + `CMD` voor de standaardargumenten.

Stap 6 — De *shell*-vorm, of de kapotte stop

Dit is het belangrijkste experiment van het lab. Bouw **dezelfde** applicatie in *shell*-vorm, op een **Ubuntu**-basis (de image `21-jre` zonder achtervoegsel):

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-debian
podman build -q -f D-shell-debian -t api-lab:shell-debian .
podman run -d --name s-deb api-lab:shell-debian
sleep 3
podman exec s-deb ps -o pid,args | head -3
```

Observeer:

```
PID COMMAND
1 /bin/sh -c java -jar /app/api.jar
2 java -jar /app/api.jar
```

De shell is PID 1 gebleven. Stop de container nu:

```
time podman stop s-deb
podman inspect --format 'code={{.State.ExitCode}}' s-deb
podman logs s-deb | tail -2
```

Observeer de waarschuwing `resorting to SIGKILL`, `real 0m10.8s`, `code=137`, en **geen** melding "SIGTERM reçu": de *shutdown hooks* zijn nooit uitgevoerd.

Vergelijk met de *exec*-vorm van stap 2:

```
podman run -d --name s-exec api-lab:1.0
sleep 3 ; podman exec s-exec ps -o pid,args | head -3
time podman stop s-exec
podman inspect --format 'code={{.State.ExitCode}}' s-exec
podman logs s-exec | tail -2
```

Observeer `1 java -jar /app/api.jar --spring.profiles.active=prod`, een stop in **0,14 s**, `code=143`, en de regels `SIGTERM reçu : arret propre en cours...` gevolgd door `API arretee proprement`.

Tot slot dezelfde *shell*-vorm, maar op een **Alpine**-basis:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar\n/app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-alpine
podman build -q -f D-shell-alpine -t api-lab:shell-alpine .
podman run -d --name s-alp api-lab:shell-alpine ; sleep 3
podman exec s-alp ps -o pid,args | head -3
time podman stop s-alp ; podman inspect --format 'code={{.State.ExitCode}}' s-alp
```

Observeer dat Java deze keer **wél** PID 1 is en dat de stop netjes verloopt (`143`).

Uitleg. De shell van busybox (Alpine) vervangt zichzelf door een eenvoudig commando; de `dash` van Ubuntu doet dat niet. **Dezelfde Dockerfile gedraagt zich dus verschillend naargelang de basisimage.** Precies het soort bug dat op de machine van de ontwikkelaar werkt en in productie breekt. De *exec*-vorm neemt de hele vraag weg.

```
podman rm s-deb s-exec s-alp
```

Stap 7 — Het opstartscript zonder `exec`

Dit patroon kom je in bedrijven het vaakst tegen:

```
printf '#!/bin/sh\neco "voorbereiding..."java -jar /app/api.jar\n' > entrypoint.sh
chmod +x entrypoint.sh
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCOPY
entrypoint.sh /entrypoint.sh\nENTRYPOINT ["/entrypoint.sh"]\n' > D-script
podman build -q -f D-script -t api-lab:script .
podman run -d --name s-script api-lab:script ; sleep 3
podman exec s-script ps -o pid,args | head -4
time podman stop s-script
podman inspect --format 'code={{.State.ExitCode}}' s-script
```

Observeer 1 {entrypoint.sh} /bin/sh /entrypoint.sh en 2 java -jar /app/api.jar, en daarna de 10 seconden en code 137 — **zelfs op Alpine**.

Corrigeer door `exec` toe te voegen:

```
printf '#!/bin/sh\neco "voorbereiding..."exec java -jar /app/api.jar\n' > entrypoint.sh
podman build -q -f D-script -t api-lab:script2 .
podman rm -f -t 0 s-script && podman run -d --name s-script api-lab:script2 ; sleep 3
podman exec s-script ps -o pid,args | head -3
time podman stop s-script ; podman inspect --format 'code={{.State.ExitCode}}' s-script
podman rm s-script
```

Observeer dat de shell verdwenen is, de stop onmiddellijk gebeurt en de exitcode 143 is.

Stap 8 — ARG is geen kluis

```
printf 'FROM docker.io/library/alpine\nARG DB_PASSWORD=leeg\nRUN echo "build met $DB_PASSWORD"
> /trace.txt\nCMD ["cat","/trace.txt"]\n' > D-arg
podman build -q -f D-arg --build-arg DB_PASSWORD='Secr3t!' -t api-lab:arg .
podman image inspect --format '{{json .Config.Env}}' api-lab:arg
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | head -3
```

Observeer dat `Config.Env` niets anders bevat dan `PATH` — tot daar klopt het, een `ARG` wordt geen `ENV` ... maar `podman history` toont `|1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build met $DB_PASSWORD" > /trace.txt`.

Uitleg. Het geheim zit in de image, leesbaar voor iedereen die ze heeft. Hoe het wél moet, komt in lab 08.

Stap 9 — USER, zijn plaats, en wat het wordt in rootless-modus

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nUSER 1000:1000\nWORKDIR /app\nRUN
mkdir /data\nCOPY api.jar /app/api.jar\nENTRYPOINT ["java","-jar","/app/api.jar"]\n' > D-user-
vroeg
podman build -f D-user-vroeg -t api-lab:user-vroeg . 2>&1 | grep -iE 'permission|error'
```

Observeer de mislukking: `mkdir: cannot create directory '/data': Permission denied`, gevolgd door `Error: building at STEP "RUN mkdir /data": exit status 1`.

Verplaats `USER` naar het einde:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nWORKDIR /app\nRUN mkdir /data &&\nchown 1000:1000 /data\nCOPY --chown=1000:1000 api.jar /app/api.jar\nUSER 1000:1000\nENTRYPOINT\n["java","-jar","/app/api.jar"]\n' > D-user-ok\npodman build -q -f D-user-ok -t api-lab:user-ok .\npodman run --rm --entrypoint id api-lab:user-ok
```

Observeer `uid=1000(...) gid=1000(1000) groups=1000(1000)`: de applicatie draait in de container niet langer als root.

Bekijk nu hoe dat er **op de host** uit ziet:

```
podman run -d --name u api-lab:user-ok ; podman run -d --name r api-lab:1.0 ; sleep 1\npodman top u user,huser,pid,hpid,comm\npodman top r user,huser,pid,hpid,comm\npodman rm -f -t 0 u r
```

Observeer:

USER	HUSER	PID	HPID	COMMAND	<- u: USER 1000 in de image
1000	100999	1	12929	java	
USER	HUSER	PID	HPID	COMMAND	<- r: root in de image
root	1000	1	13037	java	

Uitleg. `USER` geldt voor alles wat erna komt, bij de build **én** bij de uitvoering; je zet het net vóór `ENTRYPOINT`, zodra de bestanden klaarstaan. In rootless-modus is de "root" van de container al jouw eigen gebruiker (`HUSER 1000`); de UID 1000 van de container wordt daarentegen afgebeeld op `100999` — een UID uit het gereserveerde bereik van `/etc/subuid`, zonder *enig* recht op je host. `USER` blijft dus belangrijk: het ontnemt de applicatie de root-privileges *binnen* de container (imagebestanden, poorten < 1024, `apk add`), en vooral: dezelfde image draait ooit onder Docker of Kubernetes, waar root wel degelijk root is.

Opruimen

```
podman rm -f -t 0 api api2 2>/dev/null\npodman rmi api-lab:1.0 api-lab:2.0 api-lab:2.1 api-lab:3.0 api-lab:3.1 \\\n    api-lab:cmd api-lab:shell-debian api-lab:shell-alpine \\\n    api-lab:script api-lab:script2 api-lab:arg api-lab:user-ok \\\n    api-lab:alles api-lab:alles2 2>/dev/null\npodman images --format '{{.Repository}}:{{.Tag}}' | grep api-lab\npodman rmi $(podman images --filter dangling=true -q) 2>/dev/null\nrm -rf ~/labo-docker/04/node_modules ~/labo-docker/04/./gemeenschappelijk
```

Bewaar `~/labo-docker/04/api.jar` en `Api.java`: labs 05 tot 09 gebruiken ze opnieuw. Bewaar ook de images `eclipse-temurin:21-jre-alpine` en `eclipse-temurin:21-jdk`.

Wat je nu moet kunnen beweren

- De context bepaalt wat je kunt kopiëren; Podman draagt niets over, en toch blijft `.dockerignore` onmisbaar voor wat in de image belandt.
- Een vervallen instructie doet alles erna vervallen: de volgorde van de instructies bepaalt je buildtijd.
- `EXPOSE` publiceert niets.
- `ENTRYPOINT` legt het programma vast, `CMD` geeft vervangbare argumenten, en met `--entrypoint` raak je aan een debugshell.
- De *shell*-vorm kan de nette stop breken — en haar gedrag hangt af van de basisimage. Je hebt het gemeten: 0,14 s / code 143 tegenover 10 s / code 137.
- Een opstartscript moet eindigen op `exec`.
- Een `--build-arg` is zichtbaar in `podman history`.
- `USER` komt net vóór `ENTRYPOINT` — en blijft zinvol in rootless-modus.